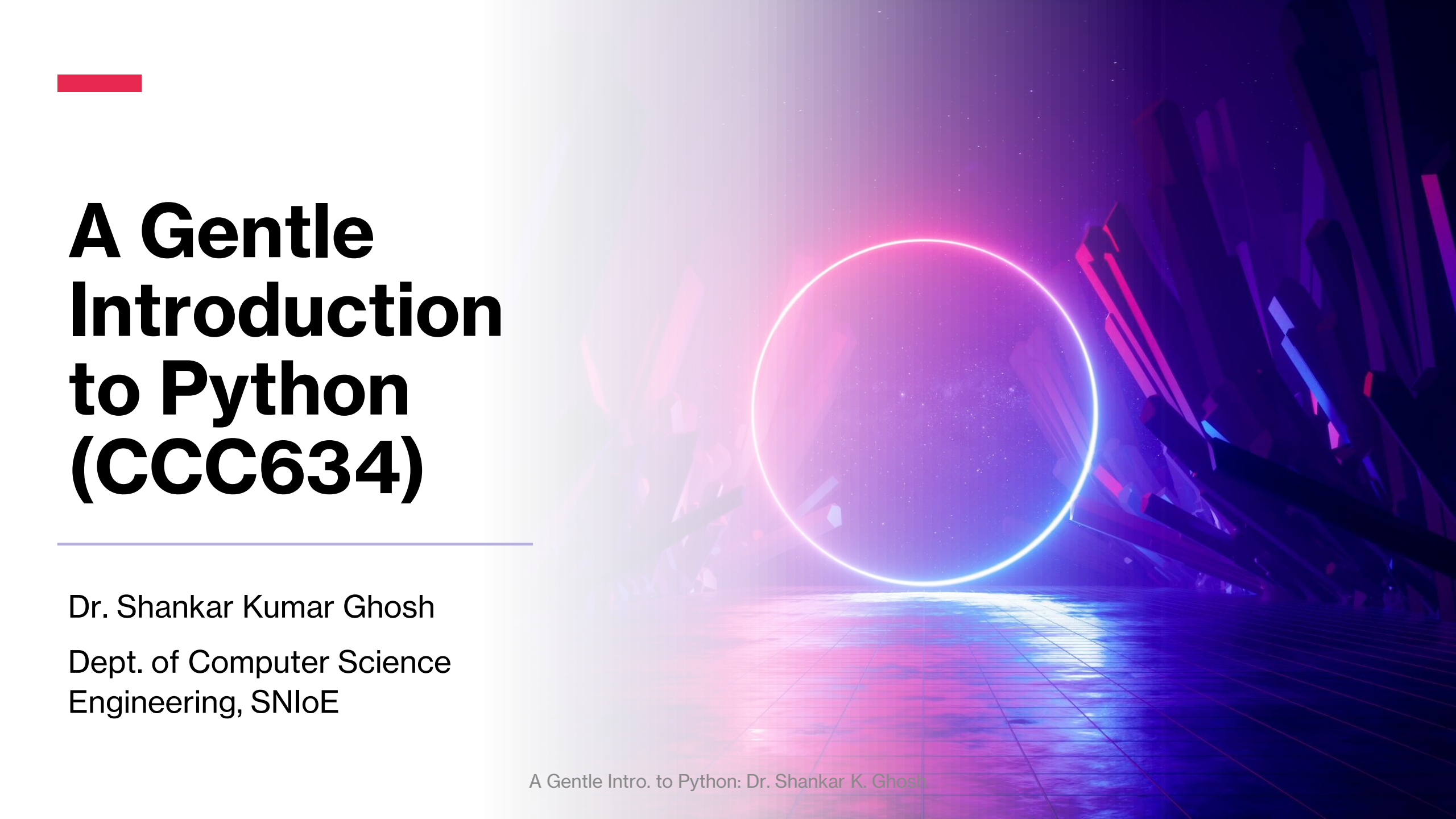




A Gentle Introduction to Python (CCC634)

Dr. Shankar Kumar Ghosh
Dept. of Computer Science
Engineering, SNioE

Reference materials

- Let us Python (5th Edition) *by* Y. Kanetkar and A. Kanetkar.
- **The Complete Reference on Python *by* Martin C. Brown.**
- Link: <https://www.python.org/>
- https://www.tutorialspoint.com/python_data_structure/python_arrays.htm

Evaluation scheme

- Midterm: 25%
- Assignment: 25%
- End term: 50%

Installation

- For windows: www.python.org/downloads (choose the appropriate one for your machine).
- For Linux: `sudo apt-get install python3.8`
- To check the version: `python3 --version`

Syllabus of the course

- Numbers
- Strings
- Dictionary
- Tuple
- List
- Modules
- Decision statements

Getting started

- Python is a specification for a language that can be implemented in different ways:
 1. Cpython: implementation in C.
 2. PyPy: Written in Rpython.
 3. Jython: Written in Java
 4. IronPython: Written in C#

All implementations are **compilers** as well as **interpreters**. The compiler converts the Python program into **intermediate bytecode**. This bytecode is then interpreted by the **interpreter**.

What is Python good for?

- ***Mathematical operations:*** standard mathematical libraries and support unlimited precision.
- ***Text processing:*** Separating the elements from a log file (time, user etc.)
- ***Rapid application development:*** Built-in modules to develop new applications.
- ***Cross-platform development:*** Deploying an application across a network.

What is Python good for?

- ***System utilities:*** Allow access to the lowest level (i.e., has access the same set of functions as operating systems).
- ***Internet Programming:*** Reading an Email/Socket programming etc.
- ***Database programming:*** SQL support.

Important packages

- ***NumPy***: Advanced mathematical operation library with support for multidimensional arrays and matrices.
- ***SciPy***: Scientific computing such as optimization, interpolation, integration etc.
- ***Pandas***: Manipulating numerical tables/time series.
- ***Matplotlib***: 2D and 3D data visualization library.
- ***OpenCV***: Open source computer vision library.

Python virtual machine (PVM)

- **PVM:** A software that emulates a physical computer.
- Programs written in Python are converted into bytecode instructions by Python compiler. Bytecode instructions remain same irrespective of **platform (microprocessor+ OS)**.
- During execution bytecode instructions are interpreted by the **virtual machine (PVM)** and then executed.

The first program (running in different ways)

- Open the Python command prompt interface and write:

```
>> print("Hello world")
```

```
>> exit()
```

- From window command prompt:

```
>> python
```

```
>> print("Hello world")
```

```
>> exit()
```

The first program (running in different ways)

- First write the code in a notepad and save it as *FirstProg.py*:

```
name = input ("Enter your name: ")  
print("Hello", name)
```

Now open the command prompt and write: `python FirstProg.py!!`

The first program (running in different ways)

Running from IDLE

- Python 3.7.9 (tags/v3.7.9:13c94747c7, Aug 17 2020, 16:30:00) [MSC v.1900 64 bit (AMD64)] on win32
- Type "help", "copyright", "credits" or "license()" for more information.
- `>>> print("hello world")`
- hello world

The first program (running in different ways)

```
===== RESTART: D:/Shankar  
Ghosh/Desktop/Semester2022JantoJun/idle1.py =====  
Enter your name: shankar  
Hello shankar
```

Components of Python program

- Python operates on *objects, statements, modules*.
- Built-in objects: *Number, String, List, Dictionary, Tuple and file*.

Integer=12345

float= 123.45

String='Hello'

List = [1, 'two', 3]

Dictionary={'one' : 1, 'two': 2}

Tuple= (1, 'two', 3)

Integer types and operators

use of integer types

print(3/4) **[Does not yeild 0]**

print(3%4) **[Modulo operator]**

Print(3//4) **[Floor division operator: Largest integer which is less than or equal to the quotient]**

Print(3**4) **[Exponent]**

Modulo and floor division operators

$$101=25*4+1$$

$101 // 4 = 25$ [Floor division operator]

$101 \% 4 = 1$ [Modulo operator]

$101/25=4.04$

Integer types and operators

```
a=10; b=25; c=15; d=30; e=2; f=3; g=5
```

[Multiple statements in a line should be separated by ;]

```
w=a+b-c
```

```
x=d**e
```

```
y=f%g
```

```
print(w,x,y) [prints values separated by spaces]
```

```
h=99999999999999
```

```
i=54321
```

```
print(h*i)
```

Float, complex and bool

use of float

i=3.5

j=1.2

print(i%j) // (2*1.2+1.1) % works on floats!!

Float, complex and bool

#use of complex

```
a=1+2j
```

```
b=3*(1+2j)
```

```
c=a*b
```

```
print(a)
```

```
print(b)
```

```
print(c) [-9+12j]
```

```
print(a.real)
```

```
print(a.imag)     It is possible to obtain real and imaginary part!!
```

```
print(a.conjugate()) [1-2j]
```

Float, complex and bool

#use of bool

```
x=True
```

```
print(a)
```

```
y=3>4
```

On evaluation of condition, it is replaced by True or False!!

```
print(x)
```

```
print(y)
```

Float, complex and bool

```
==== RESTART: D:/Shankar Ghosh/Desktop/Semester2022JantoJun/Coding/idle2.py ====
```

```
1.1
```

```
(1+2j)
```

```
(3+6j)
```

```
(-9+12j)
```

```
1.0
```

```
2.0
```

```
(1-2j)
```

```
(1+2j)
```

```
True
```

```
False
```

Built-in mathematical functions

```
print(abs(-25))
```

```
print(pow(2,4))
```

Other functions: *min, max, divmod, bin(), oct(), hex(), round()*

[Explore yourself!!]

Functions in math module

```
import math  
x=1.5357  
print(math.pi)  
print(math.sqrt(x))
```

Output:

```
==== RESTART: D:/Shankar Ghosh/Desktop/Semester2022JantoJun/Coding/math1.py ====  
3.141592653589793  
1.2392336341465238
```

Other functions: factorial, fabs, log, exp, trunc, floor, ceil [explore yourself!!]

Random numbers

```
import random
import datetime
random.seed(datetime.time())
print(random.random())
print(random.random())
print(random.randint(10,100))
```

Observation

- It is necessary to import **random** module.
- If we seed the random number generation logic with current time, we get different random numbers on each execution of the program.
- `random.seed()` with no parameter also seeds the logic with current time.

Identifying the object

- {10, 20, 30.5}
- [1, 2, 2.14, 'Nagpur']
- {12: 'simple', 43: 'complicated', 13: 'complex'}
- "Check it out"
- (3+2j)

Identifying string, list, tuple, set or dictionary

```
print(type({10, 20, 30.5}))
```

// type() is a built-in function which can determine type of any data

```
print(type([1, 2, 2.14, 'Nagpur']))
```

```
print(type({12: 'simple', 43: 'complicated', 13: 'complex'}))
```

```
print(type("Check it out"))
```

```
print(type(3+2j))
```

Identifying string, list, tuple, set or dictionary

Output:

<class 'set'>

<class 'list'>

<class 'dict'>

<class 'str'>

<class 'complex'>

String: simple example

```
#simple strings
msg1='Hoopla'
print(msg1)
msg2='He said,\"Let Us Python\".'
file1='C:\\temp\\newfile'
file2=r'C:\\temp\\newfile'
print(msg2)
print(file1)
print(file2)
```

Multiline strings

```
#multiline strings
msg3='What is this life if full of care...\n
    We have no time to spend and stare'
msg4="""What is this life if full of care...
    We have no time to spend and stare"""
msg5=('What is this life if full of care...'
    'We have no time to spend and stare')
print(msg3)
print(msg4)
print(msg5)
```

String concatenation

```
#string replication during printing
```

```
msg6='MacLearn!!'
```

```
print(msg1*3)
```

```
#Concatenation
```

```
msg7='Utopia'
```

```
msg8='Today!!'
```

```
msg7=msg7+msg8
```

```
print(msg7)
```

String: len(), min() and max()

#Use of built-in function

```
print(len('Hoopla'))
```

```
print(min('Hoopla'))
```

```
print(max('Hoopla'))
```

Observation

- Special characters can be retained in a string by either escaping them or by marking the string as a raw string.
- String cannot change, but the variables that store them can.
- **len()** returns the number of characters present in string; **min()** and **max()** return the character with minimum and maximum Unicode value from the string.

Output

D:\Sharkar Ghosh\Desktop\Semester2022JantoJun\Coding>python string1.py

Hoopla

He said,'Let Us Python'.

C:\temp\newfile

C:\\temp\\newfile

What is this life if full of care... We have no time to spend and stare

What is this life if full of care...

We have no time to spend and stare

What is this life if full of care...We have no time to spend and stare

HooplaHooplaHoopla

UtopiaToday!!

6

H

p

String: Indexing

0 1 2 3 4

28

I returned a bag of groceries

-9

-1

Indexing and slicing rules

- **String[start : end]**
- The returned string contains all of the characters starting from ***start*** up until but not including ***end***.
- If only ***start*** is specified, the slice continues until the ***end*** of the string.
- The only ***end*** is specified, the slice starts from **0** up to but not including the ***end***.

String: Slicing example

```
string = 'I returned a bag of groceries'
```

```
print(string[0])
```

```
print(string[2:10])
```

```
print(string[-1])
```

```
print(string[13:])
```

```
print(string[-9:])
```

```
print(string[:-9])
```

Output

```
D:\Shankar Ghosh\Desktop\Semester2022JantoJun\Coding>python  
string2.py
```

```
I
```

```
returned
```

```
s
```

```
bag of groceries
```

```
groceries
```

```
I returned a bag of
```

Reading a file

```
input = open('myfile.txt')  
for line in input.readlines():  
    print(line)  
input.close()
```

Array : example

#creating an array named **array1**

from array import *

array1 = array('i', [10,20,30,40,50])

for x in array1:

print(x)

i Represents signed integer of size 2 bytes

f Represents floating point of size 4 bytes

d Represents floating point of size 8 bytes

Output

```
D:\Shankar Ghosh\Desktop\Semester2022JantoJun\Coding>python  
array1.py
```

10

20

30

40

50

Array: insert() method

```
from array import *  
array1 = array('i', [10,20,30,40,50])  
array1.insert(1,60)
```

```
for x in array1:  
    print(x)
```

Array: insert() method

```
D:\Shankar Ghosh\Desktop\Semester2022JantoJun\Coding>python  
array1.py
```

```
10
```

```
60
```

```
20
```

```
30
```

```
40
```

```
50
```

Array: the remove() method

```
from array import *
```

```
array1 = array('i', [10,20,30,40,50])
```

```
#array1.insert(1,60)
```

```
array1.remove(40)
```

```
for x in array1:
```

```
    print(x)
```

Array: the remove() method

```
D:\Shankar Ghosh\Desktop\Semester2022JantoJun\Coding>python  
array1.py
```

```
10
```

```
20
```

```
30
```

```
50
```

Array: the index() method

```
from array import *  
array1 = array('i', [10,20,30,40,50])  
#array1.insert(1,60)  
#array1.remove(40)  
print(array1.index(40)) // prints 3
```

Array: update operation

```
from array import *
```

```
array1 = array('i', [10,20,30,40,50])
```

```
array1[2] = 80 // 30 will be replaced by 80
```

```
for x in array1:  
    print(x)
```

2D array

Measuring temperature of a day 4 times in consecutive 4 days

Day 1: 11 12 5 2

Day 2: 15 6 10

Day 3: 10 8 12 5

Day 4: 12 15 8 6

2D array: example

Representation:

	0	1	2	
T =	[11, 12, 5, 2],	[15, 6, 10],	[10, 8, 12, 5],	[12, 15, 8, 6]
	0	1	2	3

Accession the éléments:

```
from array import *
```

```
T = [[11, 12, 5, 2], [15, 6, 10], [10, 8, 12, 5], [12, 15, 8, 6]]
```

```
print(T[0]) // prints the first array
```

```
print(T[1][2]) // prints the 2nd element of the 1st array
```

2D array: printing the array

```
from array import *  
T = [[11, 12, 5, 2], [15, 6, 10], [10, 8, 12, 5], [12, 15, 8, 6]]  
for r in T:  
    for c in r:  
        print(c, end = " ") // To print different arrays in different lines  
    print()
```

2D array: Insertion, Updation and deletion

- Insertion: `T.insert(2, [0,5,11,13,6])` // Inserts the array in 2nd position
- Updation:
`T[2] = [11,9]`
`T[0][3] = 7`
- Deletion: `del (T[3])`

2D array: Insertion, Updation and deletion

```
from array import *  
T = [[11, 12, 5, 2], [15, 6,10], [10, 8, 12, 5], [12,15,8,6]]  
T.insert(2, [0,5,11,13,6])  
T[2] = [11,9]  
T[0][3] = 7  
del T[3]  
for r in T:  
    for c in r:  
        print(c, end = " ")  
    print()
```

Output

```
D:\Shankar Ghosh\Desktop\Semester2022JantoJun\Coding>python  
2darr.py
```

```
11 12 5 7
```

```
15 6 10
```

```
11 9
```

```
12 15 8 6
```

Container

- Contains multiple data items.
- Four types: Lists, Tuples, Sets and Dictionaries.
- List can grow and shrink during execution of the program. Hence, lists are used to handle variable length of data.
- Examples:

num= [10, 25, 30, 40, 100]

names=['Sanjay', 'Anil', 'Radha', 'Suparna']

Lists: examples

- `ages=[25, 26, 25, 27, 26]` // duplicates allowed
- `num= [10]*5` // stores `[10, 10, 10, 10, 10]`
- `lst=[]` //empty list is valid
- Accessing list elements:
`l=['shankar', 'SNU', 'cse', 'faculty']`
`print(l)`

Accessing a list

```
animals=['Zebra', 'Tiger', 'Lion', 'Jackal', 'kangaroo']
```

```
i=0 #indexing starts from 0
```

```
# using for loop
```

```
while i < len(animals):
```

```
    print(animals[i])
```

```
    i=i+1
```

```
#using while loop
```

```
for a in animals:
```

```
    print(a)
```

List operations

Mutability

```
animals=['Zebra', 'Tiger', 'Lion', 'Jackal', 'kangaroo']  
ages=[25, 26, 25, 27, 26, 28, 25]  
animals[2]='Rhinoceros'  
print(animals)  
ages[5]=31  
print(ages)  
ages[2:5]=[24, 25, 32]  
print(ages)  
ages[2:5]=[]  
print(ages)
```

List concatenation

```
lst=[25, 26, 25, 27, 26, 28, 25]  
lst=lst+[33,44,55] #concatenation  
print(lst)
```

Example:

```
D:\Shankar Ghosh\Desktop\Semester2022JantoJun\Coding>python list2.py  
[25, 26, 25, 27, 26, 28, 25, 33, 44, 55]
```

List aliasing

On assigning one list to another, both refer to the same list. Changing one changes the other. This assignment is known as shadow copy/aliasing.

```
lst1=[10,20,30,40,50]
```

```
lst2=lst1
```

```
print(lst1)
```

```
print(lst2)
```

```
lst1[0]=100
```

```
print(lst1[0], lst2[0]) # prints 100 100
```

Searching methods for Lists

An element can be searched using the **in** membership operator.

```
lst=['a', 'e', 'i', 'o', 'u']
```

```
res='a' in lst
```

```
print(res) # prints true
```

```
res='z' in lst
```

```
print(res) # prints false
```

List methods

```
lst=[12, 15, 13, 23, 22, 16, 17]
```

```
lst.append(22) # add new item at the end
```

```
lst.remove(13) #delete item 13 from the list
```

```
lst.pop() # removes last item in the list
```

```
lst.insert(3,21) # item 21 at the 3rd position
```

```
lst.count(23) # number of times 23 appears in the list
```

```
idx=lst.index(22) # prints index of the element 22
```

Sorting and reversing lists

```
lst=[10,2,0,50,4]
```

```
lst.reverse()
```

```
print(lst)
```

```
lst.sort()
```

```
print(lst)
```

Nested lists

```
a=[1, 3,5,7,9]
```

```
b=[2,4,6,8,10]
```

```
c=[a, b]
```

```
print(c[0][0], c[1][2]) # prints 1 and 6
```

List: Practice problem 1

Perform the following operations on a list of names:

- a. Create a list of 5 names: 'Anil', 'Amol', 'Aditya', 'Avi', 'Alka'
- b. Insert a name 'Anuj' before 'Aditya'
- c. Append a name 'Zulu'.
- d. Delete 'Avi' from the list.
- e. Replace 'Anil' with 'Anilkumar'.
- f. Sort all the names in the list.
- g. Print reversed sorted list.

Solution

```
R=['Anil', 'Amol', 'Aditya', 'Avi', 'Alka']
```

```
R.insert(2, 'Anuj')
```

```
R.append('Zulu')
```

```
R.remove('Avi')
```

```
i=R.index('Anil')
```

```
R[i]='AnilKumar'
```

```
R.sort()
```

```
R.reverse()
```

Output

```
D:\Shankar Ghosh\Desktop\Semester2022JantoJun\Coding>python  
prac1.py
```

```
['Anil', 'Amol', 'Anuj', 'Aditya', 'Avi', 'Alka']
```

```
['Anil', 'Amol', 'Anuj', 'Aditya', 'Avi', 'Alka', 'Zulu']
```

```
['Anil', 'Amol', 'Anuj', 'Aditya', 'Alka', 'Zulu']
```

```
['AnilKumar', 'Amol', 'Anuj', 'Aditya', 'Alka', 'Zulu']
```

```
['Aditya', 'Alka', 'Amol', 'AnilKumar', 'Anuj', 'Zulu']
```

```
['Zulu', 'Anuj', 'AnilKumar', 'Amol', 'Alka', 'Aditya']
```

List: Problem 2-adding two matrices

```
mat1=[[1,2,3], [4,5,6], [7, 8, 9]]
mat2=[[11, 12, 13], [5,5,5], [9, 11, 14]]
mat3=[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
for i in range(len(mat1)):
    for j in range(len(mat1[0])):
        mat3[i][j]=mat1[i][j]+mat2[i][j]
print(mat3)
```

Console Input/Output

Generate form of input() function is

S=input('prompt')

Prompt is a string that is displayed on the screen, soliciting a value. input() returns a string. If 123 is entered as input, '123' is returned.

input() function

input() can be used to receive 1 or more values

```
s=input('Enter your name')
```

```
print(s)
```

```
fname, mname, lname=input('Enter full name: ').split()
```

```
print(fname, mname, lname)
```

split() function will split the entered fullname with space as a delimiter. The split values will then be assigned to fname, mname, lname.

input() function

- If we are to receive multiple int values, we can receive them as strings and then convert them to ints.

```
n1, n2, n3=input('Enter three values: ').split()
```

```
n1, n2, n3=int(n1), int(n2), int(n3)
```

```
print(n1+10, n2+20, n3+30)
```

input() function

- **input()** can be used receive different types of values at a time.

```
data=input('Enter name, age, salary: ').split()
```

```
name=data[0]
```

```
age=int(data[1])
```

```
salary=float(data[2])
```

```
print(name, age, salary)
```

Output

Enter your nameshankar

shankar

Enter full name: shankar kumar ghosh

shankar kumar ghosh

Enter three values: 1 2 3

11 22 33

Enter name, age, salary: Shankar 35 1.5

Shankar 35 1.5

Practice problem

Write a program to receive radius of a circle, and length and breadth of a rectangle in one call to *input()*. Calculate and print the circumference of circle and perimeter of rectangle.

The program

```
r, l, b=input('Enter radius, length and breadth: ').split()
radius= int(r)
length=int(l)
breadth=int(b)
cir=2*3.14*radius
perimeter=2*(length+breadth)
print(cir)
print(perimeter)
```

Output

```
D:\Shankar Ghosh\Desktop\Semester2022JantoJun\Coding>python  
circle.py
```

```
Enter radius, length and breadth: 2 3 4
```

```
12.56
```

```
14
```

Tuple

- is commonly used for storing dissimilar data.

`a=()` # empty tuple

`b=(10, )` #tuple with one item

`C=('Sanjay', 25, 34)`

`D=(10, 20, 30, 40)`

`Tpl1=(10, )*5`

`Tpl2=(10)*5`

Accessing Tuple elements

- Tpl=('Sanjay', 25, 344.50)
- print(Tpl)

0 1 2 3 4

- Msg=('Handle', 'Exceptions', 'Like', 'a', 'boss')
- print(Msg[1], Msg[3])
- print(Msg[1:3])
- print(Msg[3:])
- print(Msg[: 3])

Looping in Tuples

```
tpl=(10, 20, 30, 40, 50)
```

```
i=0
```

```
while i<len(tpl):
```

```
    print(tpl[i])
```

```
    i=i+1
```

```
for n in tpl:
```

```
    print(n)
```

Tuple operations

- Tuples are immutable.

```
msg=('Fall', 'in', 'line')
```

```
Msg[0]='Fall' #error
```

```
Msg[1:3]= ('Above', 'Mark') #error
```

Tuple variates

- Nested tuple

```
a=(1, 3, 5, 7, 9)
```

```
b=(2, 4, 6, 8, 10)
```

```
c=(a, b)
```

```
print (c[0][0], c[1][2])
```

Tuple variates

- Embedded tuple

```
x=(1, 2, 3, 4)
```

```
y=(10, 20, x, 30)
```

```
print(y)
```

Unpacking a tuple

```
x=(1, 2, 3, 4)
```

```
y=(10, 20, *x, 30)
```

```
print(y)
```

Practice problem

A list contains names of boys and girls as its elements. Boys' names are stored as tuples. Write a Python program to find out number of boys and girls in the list. [Hint: use ***isinstance()*** function]

